Investigate login and file activity

Student instructions

Investigate whether repeated login failures were followed by a successful session and recorded file downloads. Complete two Python functions with ChatGPT (or the GenAI tool of your choice), verify their findings against the original logs, and document your work on GitHub. All supplied evidence is fictional.

The case and your evidence

A training evidence portal exported authentication and file-access records for 20 October 2026. Your job is to identify sessions for further review. You have 60 authentication rows and 35 file rows, including shuffled events, mixed timezone offsets, duplicate records, and malformed fields. No file contents or independent proof of identity are supplied.

Assume that both logs share the session-ID namespace. The supplied code validates rows, records exclusions, checks the CSV hashes, and normalizes timestamps to UTC. You implement the two analysis functions. Read the exact rules in their docstrings and in README.md.

Before the session

Have Python 3.10 or newer, a code editor, Git, GitHub CLI, and access to GitHub and ChatGPT (or the GenAI tool of your choice) ready. Complete steps 1 to 3 of the included GitHub helper guide before class. No extra Python packages, notebook or API key is needed.

Action	macOS or Linux	Windows PowerShell
Check Python	python3 --version	py --version
Run the analysis	python3 analyze_case.py	py analyze_case.py
Run all checks	python3 check_solution.py	py check_solution.py

If py is unavailable on Windows, try python --version. Use python only if it reports Python 3.10 or newer. In later commands, Windows users replace python3 with that working launcher.

1. Inspect the evidence and save the starting point

Extract the new student ZIP into a fresh folder. Run analyze_case.py. It should verify both hashes, write audit.json, and stop at the first TODO. A starter message here is expected.

Open both CSVs in the editor without saving changes. Find event_id, timestamp, and session_id. Inspect audit.json for an excluded row. The manifest is the supplied comparison baseline; matching it does not establish origin or completeness.

One partner follows steps 4-13 in the included GitHub helper, using a new repository named forensics-timeline-workshop. Save the ten named project files with the commit message Add workshop starter. Keep generated output out of this first commit. Both partners can work on one laptop and swap roles after function 1.

Complete the two analysis functions

2. Find successful logins after repeated failures

Use Prompt 1 from prompts.txt. Read the explanation before copying the function into analyze_case.py. Replace only find_login_sequences; keep its signature and return fields unchanged.

For each successful login at time T, count failures from the same username, source_ip, and host in the preceding five minutes. The lower bound is included; the success instant is excluded. Emit one alert when the count is at least three. The rule checks each success separately and does not reset at earlier successes.

```
T - 5 minutes <= failure time < T
```

Return the supporting failure IDs in time order. The input may be unsorted, and it must remain unchanged. Empty input returns an empty list. The loader already converts the timestamp field to a datetime object; use timedelta for comparisons.

```
python3 check_solution.py LoginChecks
python3 analyze_case.py --stage logins
```

The first command runs nine checks for function 1. The second writes login_alerts.json without requiring function 2. Explain your grouping key and one boundary comparison to your partner, then swap operator and checker.

3. Link the flagged sessions to file activity

Use Prompt 2 to implement only correlate_downloads. Match each alert to file events using username AND host AND session_id. A shared username alone is not a reliable join.

Keep only DOWNLOAD events marked SUCCESS, from the successful login time through ten minutes afterward, including both endpoints. Count events, even if paths repeat, and sum the integer bytes values. Keep sessions with zero qualifying downloads and rank the new result dictionaries by descending total bytes.

```
T <= download time <= T + 10 minutes
python3 check_solution.py DownloadChecks
python3 analyze_case.py
```

The function checks should run seven tests. The full analysis writes summary.json, containing ranked sessions and supporting event IDs. The ranking reflects recorded data volume, not a conclusion about an attacker.

If a check fails

Use the debugging prompt with your function and the actual error. Ask for the smallest correction. Keep the tests, loader, CSV files, and manifest unchanged. Rerun the analysis after every correction so you do not report stale output.

Verify the evidence and challenge the interpretation

4. Check the analysis independently

Run the full checks and record the actual result in README.md. The expected successful ending is Ran 21 tests and OK. The suite checks analysis rules, boundaries, immutability, and the supplied evidence handling.

```
python3 check_solution.py
```

Fill the data-quality and session-result tables in README.md using the outputs. Explain one rejected event using its event ID and audit reason. Rejected rows are excluded from calculations but remain in the original files; discuss how missing usable events could affect your conclusion.

For the highest-volume session, manually locate its supporting failures, successful login and matching downloads in the CSVs. Record the source, event ID, original timestamp, UTC equivalent and observation. Add the byte values yourself and compare with summary.json.

- Compare A001 with A002 for the A005 login window. Explain the exact boundary.
- Compare F003 with F004 for the A005 download window. Explain the one-second difference.
- Explain why F005 does not join to A005 even though the username matches.
- Convert one +01:00 timestamp to UTC. Check that your program compares equivalent instants.

Use event IDs when citing evidence; physical row order is deliberately shuffled. For example, cite auth_log.csv A005 rather than “the fifth line”.

5. Evaluate AI claims and write the finding

Use Prompt 3 with your summary, audit counts, and manually verified timeline. It asks the GenAI tool to review a deliberately overstated claim about identity, theft, and hash matching. Save the actual response in README.md. If the GenAI tool is already cautious, report that honestly.

Then use Prompt 4 to draft a 180- to 220-word finding. Verify every number and event ID, edit it yourselves, and identify one claim that you corrected or independently checked. Your finding must include:

- The leading session, its UTC timing and recorded download volume, with evidence IDs.
- A comparison with another flagged session and a reason for prioritization.
- Two limitations, one plausible benign explanation, and two specific next evidence requests.

A logged DOWNLOAD does not by itself identify a person, show authorization, or independently prove exfiltration. A matching hash establishes consistency with this supplied baseline, not the authenticity or completeness of the original collection. Keep observations separate from explanations.

Finish and submit the investigation

6. Record your method and save the completed work

Complete the AI-use and pair-review section of README.md, including the prompt record, a follow-up or correction, checks performed yourselves, and any instructor fallback. Both partners must explain one grouping or time-window line.

Rerun the completed analysis and full checks from the project folder. Inspect the generated output before saving the final version.

```
python3 analyze_case.py
python3 check_solution.py
```

The following Git commands are the same in PowerShell, macOS Terminal, and Linux Terminal. They assume you created and pushed the starter repository using the included helper.

```
git add analyze_case.py README.md
git add audit.json login_alerts.json summary.json
git status
```

Check that only your intended code, report, and generated analysis files are staged. The two original CSVs, manifest, helper, and tests should be unchanged.

```
git commit -m "Correlate sessions and verify forensic findings"
git push
gh repo view --web
```

Confirm that GitHub displays the completed README and current outputs. If using browser uploads instead, upload those same five changed/new files into the existing repository root and use the same commit message; complete and merge a proposed pull request if needed. Do not mix a browser-created repository with the helper's new-repository CLI sequence.

Submit one public repository link per pair

- The ten starter project files, with only the two TODO functions completed in analyze_case.py.
- audit.json, login_alerts.json, and summary.json from your final run.
- A completed README with the evidence tables, manual checks, actual test output, AI critique, and final finding.
- A starting commit and a finished commit, with private-repository access arranged as instructed.

You are finished when all 21 checks pass, the manual timeline agrees with the program, your claims are bounded by the evidence, and both partners can explain the code. An optional threshold experiment appears in README.md for pairs who finish early.